1 Introduction HKA: Integration of Digital Twins (AAS) and CAD (Siemens NX)

Within the scope of this development project, the informational and conceptual bridge between the Asset Administration Shell (AAS) and the CAD system Siemens NX is built.

> "The Asset Administration Shell is the standardized digital representation of an asset. It structures all relevant data and properties of a physical or logical object throughout its entire lifecycle, thus forming the technical foundation for the Digital Twin in Industry 4.0." [1]

The goal is to enable working with Digital Twins across system boundaries and to ensure continuous data utilization throughout the entire product lifecycle.

Methodologically, the project is divided into two consecutive phases:

1.1 Phase 1: Fundamentals and Application Scenarios in the Product Lifecycle

In the first phase of the project, the theoretical fundamentals of the AAS were developed, and the specific added value and application scenarios offered by connecting a CAD system to the administration shell were evaluated.

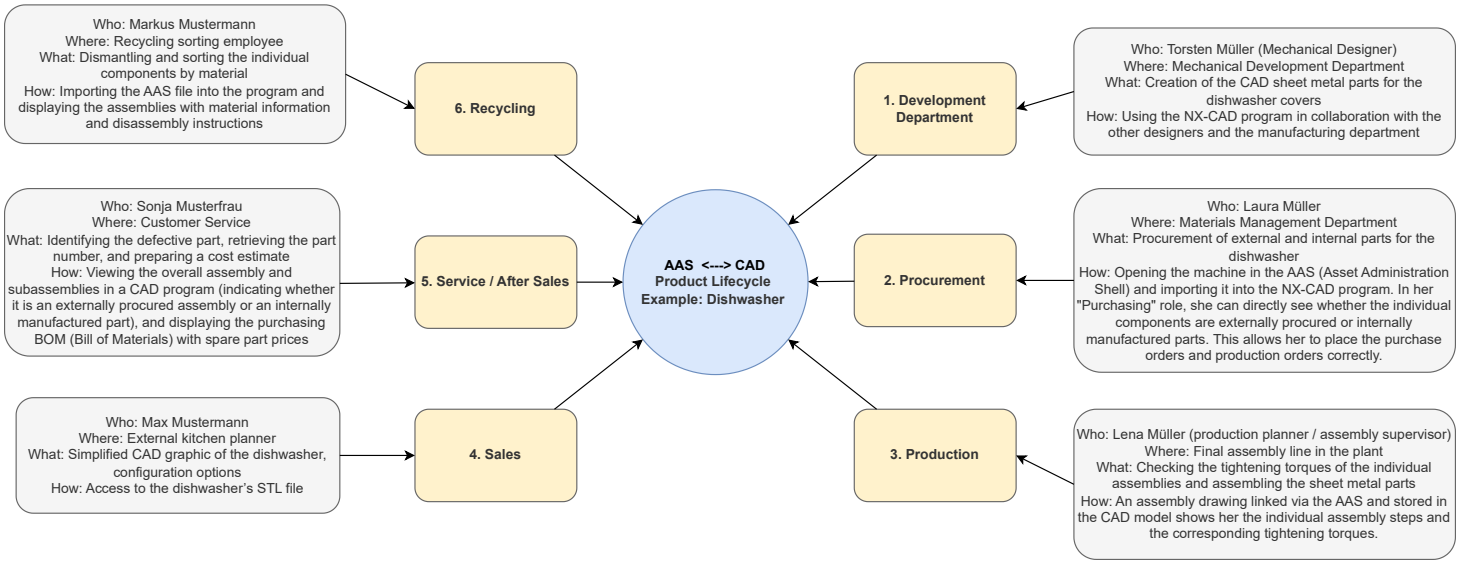
A practical example of a **dishwasher** serves to illustrate the concepts. Along its product lifecycle, highly varying requirements arise in each phase regarding the provision, updating, and structuring of data from the Digital Twin.

The considered lifecycle encompasses the following six phases:

1. Development (CAD & Engineering)
2. Parts Procurement
3. **Production (Manufacturing)**
4. Sales
5. Service / After-Sales
6. **Recycling (End-of-Life)**

*As shown in the following diagram, these phases form a continuous cycle. Within the scope of this project, the **Production** and **Recycling** phases, in particular, were elaborated in depth. The detailed*

elaboration of these use cases is the subject of the following chapters.



1.2 Phase 2: Technical Implementation and Interface Functions

Based on the conceptual preliminary work of the first phase, the focus of the second project phase was on the technical implementation. Here, the central core functions required for practical work with AASX data packages and the CAD program Siemens NX were identified and developed.

> **Documentation Note ("Living Document"):**
> Since research and work on this repository will continue in the future, this documentation is designed directly as the structure for the GitHub repository (`README.md` and linked Markdown files). Redundant, separately maintained documentation was deliberately avoided. This ensures that code changes and documentation always remain synchronous, transparently versioned, and accessible in a central location for future developers.

1.3 Scenario 3: Production (Manufacturing)

1.3.1 Detailed description of the scenario

Lena Müller works as a production planner and assembly manager in final assembly. She is responsible for the correct execution of the assembly processes as well as for the quality assurance of

the assembled modules. Lena checks the tightening torques of the individual screw connections and assembles the corresponding sheet metal components. In doing so, she ensures that all assembly specifications are strictly adhered to in order to guarantee the functionality, safety, and quality of the product.

The relevant assembly information is provided to Lena digitally. An assembly drawing, linked to the respective physical asset via the AAS, is stored in the CAD model. Using a digital device (e.g., an industrial tablet or HMI), Lena can access this information in context.

1.3.2 Process flow description

- **Product / Asset Identification:** Lena identifies the product to be assembled using a serial number or a QR code.
- **Calling up the AAS:** The corresponding AAS is loaded via the manufacturing system.
- **Accessing assembly information:** The AAS points to the linked CAD model and the corresponding assembly drawing.
- **Display of assembly steps:** The individual assembly steps are presented sequentially, including the required tools.
- **Checking tightening torques:** The target torques for each screw connection are displayed and verified with the torque tool in use.
- **Assembly of sheet metal components:** Lena assembles the sheet metal components according to the drawing and the defined process steps.
- **Documentation and feedback:** The completed assembly and the verified torques are documented in the system and, if necessary, reported back to higher-level systems.

1.3.3 Required files and resources

- **CAD models** (e.g., STEP, JT, native CAD formats)
- **Assembly drawings** (2D/3D, e.g., PDF or directly in CAD)
- **Torque specifications** (e.g., tables or metadata)
- **Asset Administration Shell** (AAS model in JSON or XML format)
- **Digital devices** (tablet, industrial PC, HMI)
- **Torque tools** (ideally with a digital interface)
- **MES/PLM systems** for data management

1.4 Technical implementation

1.4.1 Implementation in the Asset Administration Shell

The AAS serves as a central data hub for assembly- and quality-relevant information. Technically, the implementation is carried out via clearly defined `SubmodelElements` that standardize assembly knowledge, CAD references, and torque specifications.

Assembly and torque data

- **Submodel "AssemblyInformation":** Contains the complete description of the assembly steps as structured `SubmodelElements` (e.g., `Operation`, `Sequence`, `StepNumber`).
- **Tightening torques:** Each screw connection is described by a `Property` element:
  - `FastenerID`
  - `TorqueNominal` (e.g., 12 Nm)
  - `TorqueTolerance` (e.g., ±10%)
  - `ToolType` (e.g., electronic torque wrench)

> **Example:** The screw connection of the side panel has the attribute `TorqueNominal = 12 Nm`, `TorqueTolerance = ±1.2 Nm`.

1.4.2 Implementation in the CAD environment

The CAD environment (viewer or assembly frontend at Lena's workstation) acts as a visual and interactive interface between the worker and the digital product twin.

- Each component and screw connection in the CAD model is linked to the `InstanceId` of the corresponding AAS component.
- If Lena clicks on a screw in the 3D model, the **target torque**, **tolerance**, and **tool type** are loaded directly from the AAS and displayed.

Feedback and reporting channel (As-Planned vs. As-Built)
Lena can report deviations directly in the CAD interface, e.g.:

- Screw is inaccessible
- Component does not fit stress-free
- Torque cannot be achieved structurally

These reports are stored as an `Event` or `Annotation` directly in the AAS and are made available to engineering and production planning.

1.5 Scenario 6: Recycling (End-of-Life)

1.5.1 Detailed description of the scenario

In 2026, the linear economy ("take, make, dispose") was legally ended by the *Ecodesign for Sustainable Products Regulation* (ESPR). Every dishwasher sold in the EU must now have a Digital Product Passport (DPP), which is technically implemented as an AAS.

Markus Mustermann, a specialized employee in a certified dismantling center, faces a challenge: A modern dishwasher consists of over 150 different materials, including valuable stainless steel, copper-bearing motors, electronics with rare earths, and problematic bitumen insulation mats.

Without the AAS, the dishwasher would be a "black box" for Markus. He would have to shred it, which severely degrades material purity. Thanks to the AAS, Markus becomes an Urban Miner: He accesses the digital twin, which tells him not only what is installed but also how to separate it non-destructively to maximize recycling yields.

1.5.2 Process flow description

The workflow is divided into five phases:

1. **Capture & Initialization:** Markus scans the QR code lasered onto the housing frame. The AAS opens the corresponding model for this.
2. **Status & Safety Analysis:** Before Markus uses any tools, the system checks the "Safety" submodel. It displays whether capacitors need to be discharged or if chemical residues (e.g., rinse aid remnants) are present in the hoses.
3. **Virtual Exploration:** Markus receives the dismantling drawings in the AAS and disassembles the machine based on them. He is shown which part is made of which material. At the end, he enters how much mass of each material he has separated by type.
4. **Guided Dismantling:** Markus follows the steps stored in the AAS. The system dictates:
    - *Step 1:* Loosen the Torx screws (T15) on the back panel.
    - *Step 2:* Disconnect the wiring harness.
    - *Step 3:* Remove the insulation.

1.5.3 Required files and resources

The following items are required for the implementation:

- **The AASX container file:** Contains the logical structure and all references.
- **Identification:** Unique ID.

- **Digital Product Passport (DPP):** Contains the legally required ESPR data (carbon footprint, recyclate content).
- **Bill of Materials (R-BOM):** The hierarchical list as well as properties for recycling all components.
- **TechnicalProperties:** Material data sheets per component, provided by the respective manufacturers and merged into the dishwasher's AAS.
- **CAD resources:** Dismantling drawings and 3D model for disassembly.

1.6 Technical implementation

1.6.1 Implementation in the Asset Administration Shell (AAS)

The AAS serves as a data hub. All installed components have their properties stored by the respective manufacturers, and these are assembled into a dishwasher.

- **Material Identification:** Every component in the dishwasher receives a `Property` element with the material ID.
  - *Example:* The cladding elements receive the attribute `Material: V2A (Stainless Steel)`.
- **API Interface:** The AAS provides a REST API through which the recycling software makes queries (e.g., `GET /submodels/MaterialComposition/submodel-elements/CopperWeight`).

1.6.2 Implementation in the CAD environment

The CAD environment serves as the central data source for creating the AAS. The process is structured as follows:

- **Provision of documents:** Dismantling drawings in PDF format are generated directly from the CAD system and made available to the AAS.
- **Material flow analysis:** An automated list is generated that records all components, including their volumes and materials. From this, the exact masses of the different raw materials can be derived.
- **Target-actual comparison in the recycling process:** As part of the recycling process, this data is calculated for each dishwasher variant and stored in the AAS as target values.
- **Proof of obligation:** After dismantling, the recycler is obliged to enter the actually recovered masses of the different materials as actual values into the AAS.
- **Feedback loop to the manufacturer:** This data is fed back to the manufacturer. This gives the manufacturer seamless proof of the percentage to which the machine was sorted and dismantled.
- **Visualization:** Given high system complexity, an additional 3D model can be provided to the recycler.
- **Additional visualization:** The system calculates the recycling priority based on the stored material properties. Components containing pollutants are displayed flashing red.

1.6.5 Regulations and Legal Guidelines (Additional Requirements)

1. **ESPR (Ecodesign for Sustainable Products Regulation)**
   - **Standard tools & dismantlability:** The existing Regulation (EU) 2019/2022 for dishwashers already stipulates that certain parts (such as pumps or motors) must be replaceable with "commonly available tools" and without causing permanent damage to the appliance.
   - **Dismantling time:** The new ESPR (Regulation (EU) 2024/1781) allows the EU to set explicit performance requirements in product-specific "delegated acts". For the revision of the dishwasher rules (planned from 2026), discussions are underway to include dismantling time as a metric for recyclability [2], [3].
2. **EU Battery Regulation (2023/1542)**
   - **Removability:** From February 18, 2027, portable batteries must be designed so that they can be easily removed and replaced by the end user or qualified personnel.
   - **Battery passport:** While the full "battery passport" primarily applies to industrial (>2 kWh) and traction batteries, information on chemical composition and safe removal for small batteries must be stored in the Digital Product Passport (DPP) of the main device (here, the dishwasher's AAS) [4].
3. **WEEE Directive (Waste Electrical and Electronic Equipment)**
   - WEEE regulates the handling of electronic waste after use.
   - **Collection and recovery targets:** For large appliances (Category 1), which include dishwashers, the directive prescribes a recovery target of 85% and a target for preparation for reuse and recycling of 80%.
   - **Role of the AAS:** The AAS is a tool to guarantee "material hygiene". These high targets can only be achieved economically through single-origin separation (e.g., grade 1.4301 stainless steel instead of mixed scrap) [5].
4. **Substances of Concern (REACH & SCIP)**
   - **Information obligation:** Since January 2021, manufacturers must submit information on Substances of Very High Concern (SVHC) to the ECHA SCIP database if the concentration exceeds 0.1%.
   - **AAS Integration:** The AAS serves as an interface to make this complex chemical data directly visualizable for Markus Mustermann at his workstation (e.g., warning label for bitumen mats or flame retardants in circuit boards), without him having to search manually in databases [6], [7].

2 Introduction GitHub

AAS-CAD Integration: Connection between Asset Administration Shell and CAD

Introductory document for our AAS-CAD Integration project.

In this project, we are working on enabling the Asset Administration Shell (AAS) – the standardized digital twin – to communicate directly with our CAD systems. The goal is to enable bidirectional data exchange and to provide concrete code examples and workflows for this purpose.

2.1 What exactly is this about?

The current state of the art is: 3D models reside in CAD, and the digital twin lives completely isolated from it in its own software environment. However, if we want to implement use cases like intelligent recycling or seamless material tracking, these two worlds must talk to each other. This repository collects scripts and plug-ins to build exactly this bridge.

2.2 Core Features Overview

We focus on four fundamental functions to establish the interaction between AAS and CAD. In the table, you will find direct links to the respective explanations and implementations for Siemens NX and PTC Creo, as well as the current development status for Autodesk Fusion 360.

#	Feature	Implementation in Siemens NX	Implementation in PTC Creo	Implementation in Autodesk Fusion 360
1	**Import of 3D data from the AAS** *(Loading the model*	NX Part Loader	Creo Doc	🚧 Work in Progress

#	Feature	Implementation in Siemens NX	Implementation in PTC Creo	Implementation in Autodesk Fusion 360
	from the digital twin into CAD)			
2	**Assignment of an Asset ID to a part** *(The fundamental link between the 3D model and the AAS instance)*	NX Asset-ID assign	Creo Doc	🚧 Work in Progress
3	**Synchronize properties** *(Exchanging material data, metadata, etc.)*	NX Sync Properties with AAS	Creo Doc	🚧 Work in Progress
4	**Data export from CAD** *(currently as CSV)*	NX to AAS Export	Creo Doc	🚧 Work in Progress

2.3 Detailed Description of the Features

How exactly these four functions work in detail and how they are solved on the system side can be found directly in the linked documents from the table above. There you will find detailed explanations and code examples for Siemens NX and PTC Creo.

2.4 Contributing

We always welcome support. If you want to contribute your own scripts, have a solution for another CAD system like SolidWorks, or find bugs, feel free to create a Pull Request.

2.5 License

The core project "Connecting CAD to Digital Twins" is licensed under the **Apache License 2.0**. Detailed information can be found in the LICENSE.md file.

> **Important note regarding Siemens NX (UI-Styler):** > This project contains UI code generated with the **Siemens NX 2021 UI-Styler** and manually adapted. To compile or run this code, proprietary NX Open libraries (DLLs) are required, which are **not** part of this repository. The rights to these libraries and to the basic framework of the generated code remain with Siemens AG. Please observe the corresponding license terms (EULA) of your Siemens NX installation.

Further legal notices are listed in the NOTICE.md file.

3 Prerequisites

Before setup, make sure to download the required scripts from this repository.

For the successful setup and execution of the integration between Siemens NX and the AAS, the following system requirements must be met:

3.1 Software & Licenses

- **Siemens NX:** Local installation (e.g., version 2021).
- **NX License:** Enterprise or NX Open license for executing and compiling scripts and external programs.
- **AASX Package Explorer:** Tool for viewing and validating the AAS.

3.2 Development Environment & Languages

- **Python:** Standalone installation (version 3.11 or higher) for adapters, because the Python version in NX 2021 is only version 3.8 and does not include the lxml library.
- **IDE:** Visual Studio (e.g., for C#) or Visual Studio Code (for Python) to customize the scripts.

3.3 System Rights & Configuration

- **Administrator Rights:** Local Windows admin rights (for installations via `pip` and setting environment variables).

- **NX Role:** An advanced user role (e.g., "Advanced") to unlock the developer tab in the NX user interface.

4 Function Descriptions

The following sections describe the implementation of the core features.

4.1 NX AASX to STEP Import Tool

This project contains a Python-based workflow for importing CAD geometry from an **AASX file (Asset Administration Shell package)** into **Siemens NX**.

The solution consists of two scripts:

- **NX launcher script**
  Executes inside Siemens NX via the **NX Open Python API**
- **External AAS script**
  Extracts a STEP file from an AASX package

The workflow connects an AAS with a CAD system by extracting a referenced STEP model and importing it into NX as a native `.prt` component.

4.2 Overview

The tool enables a semi-automated workflow to extract and load CAD data from an AASX container into Siemens NX.

4.2.1 Key Features

- Interactive execution from Siemens NX
- Launch of an external Python extraction script via `subprocess`
- Automatic detection or creation of an active NX work part
- Conversion from **STEP** to **NX PRT**
- Insertion of imported `.prt` files as assembly components

- Support for multiple component instances
- Logging and error output via the NX Listing Window

4.3 Basic Functionality

The current implementation performs the following workflow:

1. The NX launcher script starts inside Siemens NX
2. The launcher executes the external script `AAS_TO_NX.py`
3. The external script handles:
    - AASX file selection
    - STEP model identification
    - STEP extraction
4. The NX launcher checks whether the STEP file was created
5. The STEP file is converted into a native NX `.prt` file
6. The resulting `.prt` file is inserted into the active assembly as a component
7. A second component instance can be added at another position

4.4 Project Structure

The solution consists of two scripts:

- **NX launcher script**
- **External extraction script**: `AAS_TO_NX.py`

4.5 NX Launcher Script

The NX launcher script is responsible for the NX-side import workflow.

It currently includes the following main tasks:

- Ensure that a valid work part exists
- Launch the external extraction script

- Convert STEP to NX PRT
- Insert the imported geometry as assembly components
- Write status and error information to the NX Listing Window

`main()`

Entry point of the NX launcher script.

Responsibilities

- Opens the NX Listing Window
- Resolves the path to the external script
- Deletes an old STEP file before starting the extraction
- Executes the external script via `subprocess.run(...)`
- Evaluates return code, stdout, and stderr
- Verifies whether the extracted STEP file exists
- Ensures that an NX work part is available
- Imports the STEP file as one or more assembly components

Notes

- The launcher currently checks for `temp_model.step` in `AAS-Creo-Bridge/temp_model.step`
- It also tries to place a second instance using `temp_model.step` in the launcher base directory
- The code currently validates only the first STEP path before import

`ensure_work_part()`

Ensures that an active NX work part exists.

Behavior

- Checks whether `session.Parts.Work` is available
- If no work part exists:

- Looks for `StartPart.prt` in the launcher directory
- Opens it if it already exists
- Otherwise creates a new part in millimeter units

Purpose

This prevents NX Open operations from failing due to a missing active work part.

`identity_matrix()`

Creates and returns an identity rotation matrix of type `NXOpen.Matrix3x3` .

Purpose

Used as the default orientation matrix when inserting components.

`rotation_matrix_x(angle_deg)`

Creates a rotation matrix for a rotation about the **X axis**.

Input

- `angle_deg` : rotation angle in degrees

`rotation_matrix_y(angle_deg)`

Creates a rotation matrix for a rotation about the **Y axis**.

Input

- `angle_deg` : rotation angle in degrees

`rotation_matrix_z(angle_deg)`

Creates a rotation matrix for a rotation about the **Z axis**.

Input

- `angle_deg` : rotation angle in degrees

`get_rotation_matrix(axis="X", angle_deg=0.0)`

Returns a rotation matrix depending on the selected axis.

Supported axes

- `X`
- `Y`
- `Z`

If the axis is invalid, the function returns the identity matrix.

Note

This helper function exists in the code, but the current component insertion logic does **not yet apply** the computed rotation matrix.
`add_prt_as_component()` currently uses `identity_matrix()` directly.

`import_step_to_prt(step_path)`

Converts a STEP file into a native NX `.prt` file.

Behavior

- Creates an NX `Step242Importer`
- Uses native file system import mode
- Imports the file into a **new part**

- Enables import of:
  - Curves
  - Surfaces
  - Solids
  - PMI data
- Writes the output `.prt` file next to the STEP file

Returns

- Path to the generated `.prt` file on success
- `None` on failure

`open_part(part_path)`

Opens an NX part file using `session.Parts.OpenBaseDisplay(...)`.

Behavior

- Opens the specified `.prt` file
- Logs success or failure in the NX Listing Window

Returns

- The opened part object on success
- `None` on failure

Note

This function is currently available in the launcher code but is not used by `main()`.

add_prt_as_component

Adds a `.prt` file as a component to the target assembly.

Behavior

- Creates a placement point from `x`, `y`, `z`
- Uses the NX `ComponentAssembly.AddComponent(...)` API
- Assigns the given component name
- Inserts the part as `"Entire Part"`

Returns

- The created component on success
- `None` on failure

Important Note

Although the function signature includes:

- `axis`
- `angle_deg`

the current implementation does **not** apply rotational placement.
The component orientation is currently always the identity matrix.

import_step_into_nx_as_component

Central import helper for NX-side processing.

Process

1. Ensure an active work part exists
2. Convert STEP to PRT
3. Derive a default component name from the `.prt` filename if needed
4. Insert the `.prt` file as a component into the assembly

Returns

- The created NX component on success
- `None` on failure

Multiple Component Placement

The NX launcher currently supports inserting multiple instances of the imported model.

Current Example in Code

- `Part_1` at `(0.0, 0.0, 0.0)`
- `Part_2` at `(150.0, 0.0, 0.0)`

This allows simple assembly population with repeated geometry.

Note

The second import currently references a different STEP path:

- `AAS-Creo-Bridge/temp_model.step`
- `temp_model.step` in the base directory

This behavior should be kept in mind when testing the workflow.

External Script: `AAS_TO_NX.py`

The external script is responsible for extracting a STEP file from the selected AASX package.

Its functionality remains unchanged.

`main()`

Entry point of the external extraction script.

Process

1. Opens a file selection dialog
2. User selects an `.aasx` file
3. The AASX content is analyzed
4. A matching STEP file is extracted

5. The STEP file is written to a temporary location

`select_aasx_file()`

Opens a file dialog using Tkinter.

Behavior

- Allows the user to select an `.aasx` file
- Aborts the process if no file is selected

`ask_part_name()`

Prompts the user to enter a specific part name.

Behavior

- Uses a Tkinter input dialog
- Validates that the input is not empty
- Defines which STEP model should be extracted from the AASX package

`extract_step_from_aasx_by_name()`

Performs a targeted extraction of a STEP model from the AASX package.

Process

1. Open and parse the AASX file
2. Iterate over available AAS shells
3. Inspect referenced files and metadata
4. Identify STEP files
5. Compare the file name with the user-defined part name
6. Extract the matching STEP file

7. Save it to a temporary location
8. Return the resulting STEP path

Returns

- STEP file path if a matching file is found
- `None` if no matching STEP file exists

4.6 Installation and Execution in NX

1. Ensure Python is installed and available
2. Ensure the NX Open Python API is available
3. Place both scripts in the expected project structure
4. Open Siemens NX
5. Execute the NX launcher script

The launcher then starts the extraction and import workflow automatically.

4.7 Troubleshooting

No file selected

Possible cause:

- The AASX file selection dialog was cancelled

No matching STEP file found

Possible causes:

- No STEP file matches the entered part name
- Naming mismatch between AASX file content and user input

External script failed

The launcher reports the external return code and any stdout/stderr output in the NX Listing Window.

Possible causes:

- Missing Python environment
- Invalid script path
- Runtime error in the external script

STEP file was not created

Possible causes:

- Extraction failed
- Output path mismatch
- File was written to a different temporary location than expected

Import into NX failed

Possible causes:

- STEP to PRT conversion failed
- NX importer configuration issue
- Invalid or inaccessible file path
- Component insertion failure in the current assembly

4.8 Possible Improvements

- Apply the computed rotation matrix during component placement
- Validate both STEP paths before inserting multiple instances
- Use a consistent temporary output path
- Let the user select from available STEP models instead of entering a name manually
- Support import of multiple different components from one AASX package
- Add automatic placement using metadata

- Add configurable paths instead of fixed relative paths
- Write logs to a file in addition to the NX Listing Window

5 NX addPartID for AAS

This project includes a Siemens NX Open application (C#) created using the NX Block UI Styler. For detailed instructions and technical background on working with this tool, please refer to the official Siemens manual **"NX Open Programmer's Guide"**, specifically the chapters on the **"Block UI Styler Introduction"** (for dialog creation and callbacks).

The tool enables designers to assign a unique ID (Part ID) to an active NX part via a simple graphical user interface. This ID is stored as a part attribute (`PART_ID`) and is used to link the 3D model with the AAS for digital twins.

5.1 Basic functionality (Adding the Part ID for the AAS)

To make a 3D model usable in Industry 4.0 scenarios, it must be uniquely identifiable. This script addresses this requirement by providing a defined workflow in NX:

1. **User input:** The user starts the script in NX. A dialog box (defined by the `.dlx` file) opens, in which the desired ID for the AAS is entered.
2. **Validation:** The tool verifies whether an ID has been entered and whether a part is opened and active (Work Part) in NX.
3. **Attribute assignment:** The entered string is written directly into the part metadata via the NX Open API. The attribute **`PART_ID`** is created or overwritten.

Once the part is saved and, for example, exported as a STEP file (including metadata), the AAS can read this `PART_ID` and associate the part with a digital twin.

5.2 Code structure and the individual functions

The code is based on the standard template of the NX Block UI Styler. Below is a detailed description of the key functions in `add_PartID.cs` :

`Main()`

The entry point of the application. A new instance of the `add_PartID` class is created, and the dialog is displayed on the user's screen via the `Show()` method.

`initialize_cb()`

(Initialize Callback)
This method is executed immediately when the dialog is loaded. It establishes the connection between the C# logic and the graphical user interface (the `.dlx` file). Here, the input field (block type: String) is located in the UI and assigned to the variable `string0` , so it can be accessed later in the process.

`apply_cb()`

(Apply Callback)
This function is triggered when the user clicks **"Apply"**:

- **Read values:** Retrieves the text from the input field (`string0.Value`).
- **Validation checks (If-Statements):**
  - Checks whether the field is empty. If so, an informational message is displayed and the process is stopped.
  - Checks whether an active work part (`workPart`) exists. Without a part, no attribute can be assigned.
- **Write operation:** The command `workPart.SetAttribute("PART_ID", eingegebeneID);` writes the actual attribute to the NX part.
- **Session Undo:** Creates a visible undo step ("Part ID set"), to keep the NX session clean.

`ok_cb()`

(OK Callback)
Executed when the user clicks the "OK" button. This function internally calls `apply_cb()` , thereby performing the attribute assignment and then closing the dialog window.

`GetUnloadOption()`

Defines how the program is handled in NX memory after execution. Here, `Session.LibraryUnloadOption.Immediately` is set. This means that memory is released immediately after the script has been executed. This prevents memory leaks during an NX session.

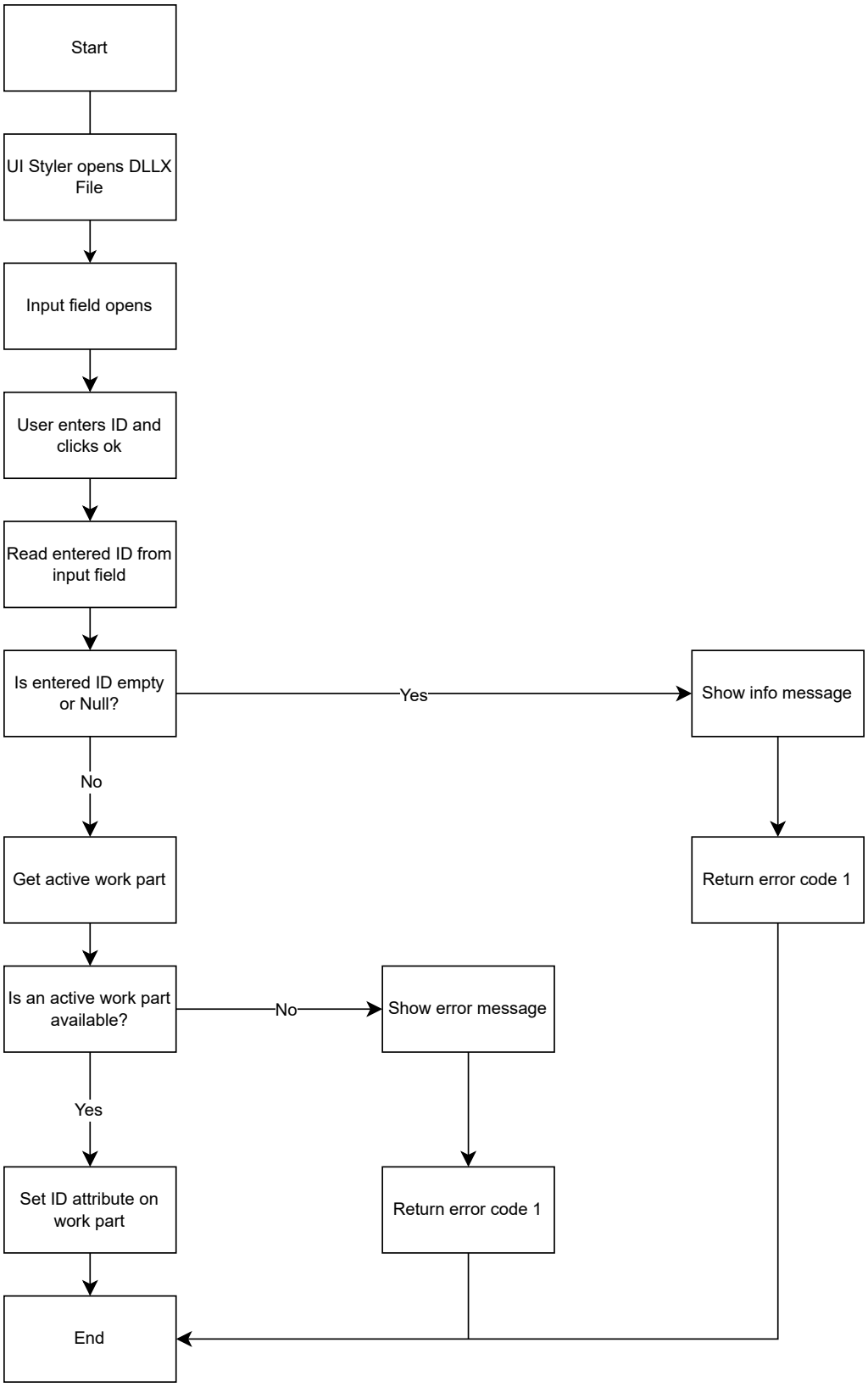
5.3 The User Interface (`add_PartID.dlx`)

The included XML file (`.dlx`) defines the visual appearance of the NX dialog. It includes:

- The window title: **"Add Part ID"**
- A group (`group0`) for visual structuring
- The essential text input field (`string0`) labeled **"Enter Part ID"**.

5.4 Technical workflow diagram

The following Diagram shows a **flowchart** that describes the technical functionality of the implemented process.



5.5 Installation & Execution in NX

1. Download the files `add_PartID.cs` and `add_PartID.dlx`
2. Ensure that both files are located in the same directory or placed according to the NX environment variables (e.g., in the `application` folder under `UGII_USER_DIR`)
3. Open a part in Siemens NX.
4. Execute the C# file via (File -> Execute -> NX Open) or integrate it as a button into your NX user interface (Ribbon).

5.6 Troubleshooting

- **Error during startup (dialog cannot be found):** By default, the C# script automatically locates the `add_PartID.dlx` file within the same directory where the script is stored. There is no need to manually adjust the path in the code, as long as the .cs file and the .dlx file remain in the same folder. Alternatively, the `.dlx file` can be placed in a standardized NX directory `($UGII_USER_DIR/application/)` for automatic detection by NX.".
- **Message "No active part available":** The script was executed without a part being open, or the displayed part is not the "Work Part". Set the part as the Work Part by right-clicking it in the Assembly Navigator and try again.

6 NX Add Attributes Tool

This project includes a Siemens NX Open application (Python) created using the NX Block UI Styler. For detailed instructions and technical background on working with this tool, please refer to the official Siemens manual **"NX Open Programmer's Guide"**, specifically the chapters on the **"Block UI Styler Introduction"** (for dialog creation and callbacks).

The tool enables designers to assign **custom user-defined attributes** (name + value) to the active NX part via a simple graphical user interface.

6.1 Basic functionality (Adding custom attributes to an NX part)

In many engineering workflows, additional metadata must be attached to a CAD model (e.g., IDs, descriptions, classifications). This script provides a flexible way to assign such attributes:

1. **User input:**
   The user starts the script in NX. A dialog box (defined by the `.dlx` file) opens with two input fields:
   - Attribute name (Label/Title)
   - Attribute value
2. **Validation:**
   The tool checks:
   - Whether a part is currently open and active (Work Part)
   - Whether a valid attribute name has been entered (not empty)
3. **Attribute assignment:**
   The entered attribute is written directly into the part metadata using the NX Open API.
   The attribute is created or overwritten dynamically based on the user input.

This allows flexible enrichment of NX parts with metadata for downstream processes such as PLM integration, export, or digital twin applications.

6.3 Code structure and the individual functions

The code is based on the standard template of the NX Block UI Styler. Below is a detailed description of the key functions in `Add_Attributes.py`:

`main()`

The entry point of the application.
An instance of the `Add_Attributes` class is created, and the dialog is displayed using the `Show()` method. After execution, resources are properly released via `Dispose()`.

`_init_()`

(Constructor)
Initializes the NX session and UI, loads the `.dlx` dialog file, and registers all callback functions:

- Apply
- OK
- Update
- Initialize
- DialogShown

`initialize_cb()`

(Initialize Callback)
Executed when the dialog is loaded.
This function connects the UI elements (defined in the `.dlx` file) with the Python variables:

- `string0` → Attribute name
- `string01` → Attribute value

These references allow access to user input later in the process.

`dialogShown_cb()`

(Dialog Shown Callback)
Executed just before the dialog is displayed.
Currently not used, but can be extended to:

- Set default values
- Pre-fill fields
- Configure dynamic UI behavior

`apply_cb()`

(Apply Callback)
This is the **core function** of the application and is triggered when the user clicks **"Apply"**:

- **Read values:**
  Retrieves the input from both text fields:
  - Attribute name (`string0`)
  - Attribute value (`string01`)
- **Validation checks:**
  - Ensures a Work Part exists
  - Ensures the attribute name is not empty
- **Write operation:**
  The attribute is written to the active part using:

  ```
  workPart.SetUserAttribute(...)
  ```

- **Error handling:**
  If any issue occurs, an error message is displayed via the NX Message Box.

`update_cb(block)`

(Update Callback)
Triggered whenever a UI element changes.
Currently not used, but can be extended for:

- Live validation
- Dynamic UI updates
- Enabling/disabling fields

`ok_cb()`

(OK Callback)
Executed when the user clicks the **"OK"** button.
Internally calls `apply_cb()` to perform the attribute assignment and then closes the dialog.

`GetBlockProperties(blockID)`

Helper function to access the properties of UI elements (blocks).
Used to retrieve values from the dialog input fields.

`Dispose()`

Ensures proper cleanup of the dialog and frees NX resources after execution.

The User Interface (`Add_Attributes.dlx`)

The included XML file (`.dlx`) defines the visual structure of the dialog:

- Two input fields:
  - **Attribute Name** (`string0`)
  - **Attribute Value** (`string01`)
- Standard NX dialog buttons:
  - Apply
  - OK
  - Cancel

The UI is intentionally minimalistic to allow fast and flexible attribute entry.

6.4 Installation & Execution in NX

1. Download the files `Add_Attributes.py` and `Add_Attributes.dlx`
2. Ensure both files are located in the same directory
   (or in a configured NX application directory, e.g. `$UGII_USER_DIR/application`)
3. Open a part in Siemens NX
4. Run the script via:
   - **Menu:** File → Execute → NX Open → Python
   - Or integrate it into the NX Ribbon

6.7 Troubleshooting

- **Dialog cannot be found:**
  Ensure that the `.dlx` file is located in the same directory as the Python script. The script automatically resolves the path based on its own location.
- **Error: "No active work part found":**
  No part is currently active. Open a part or set an existing part as the Work Part.
- **Error: "Attribute name cannot be empty":**
  The first input field must contain a valid attribute name.

7 Attribute Export Skript

This Python script uses the NXOpen Python API to read specific attributes of an open NX part and export them into a CSV file.

The CSV file contains selected mass properties of a part, e.g.:

- Volume
- Mass
- Surface area
- Material

The file is automatically saved in the same directory as the NX part.

7.1 Basic Functionality

The script executes the following process:

1. Establish a connection to the current NX session
2. Access the active work part
3. Check whether the active part is a standalone part or an assembly
4. Define the standard attributes to be exported
5. Update the mass properties of each relevant part
6. Read mass, volume, and area directly via NX measurement functions

7. Read additional standard attributes, such as material, from part attributes
8. Collect all custom user attributes from the part
9. In assemblies, additionally collect instance-specific attributes from each leaf component
10. Merge part attributes and component attributes, with component attributes overriding identical names if necessary
11. Remove standard attributes from the custom attribute set to avoid duplicate CSV columns
12. Create one CSV row per standalone part or per leaf component in an assembly
13. Build a sorted CSV header containing standard and custom attributes
14. Write the extracted data to a CSV file

7.2 Code Structure and Functions

main()

The entry point of the script.

Functions:

- Initialize Session: Access the current `NXOpen.Session`.
- Get Active Part: Access the `workPart` (abort if none is open).
- Determine Script Directory: Use `Path(__file__)` to get the folder where the script is stored.
- Define Target Attributes: Specify required keys (e.g., `MassPropMass`, `Material`).
- Update Mass Properties: Use `MassCalculationBuilder` to ensure geometry data is up-to-date.
- Collect Attribute Values: Iterate through all part attributes and filter by defined keys.
- Define CSV Path: Combine the script directory with the part name for the export file.
- Export to CSV: Write the filtered values into a `.csv` file located in the script's folder.

Definition of the required attributes

The following attributes are exported:

wanted_attributes = [
"MassPropVolume",
"MassPropMass",
"MassPropArea",

"Material"
]

These attributes correspond to mass properties of an NX part.

Attribut	Description
MassPropVolume	Volume of the part
MassPropMass	Mass of the part
MassPropArea	Surface area of the part
Material	Material of the part

Creation of the export file path

The file path of the CSV file is generated automatically

7.3 Sync other Properties

In addition to the standard and user-defined attributes, the script also supports attributes that were added using the **"NX Add Attributes"** functionality.

These attributes are automatically detected during runtime and included in the export process without requiring any additional configuration or mapping.

This ensures that all relevant attribute data—whether originally defined in the part or subsequently added via NX—is consistently captured and exported to the CSV file.

7.4 Troubleshooting

No CSV file is generated
Possible causes:

- No active work part is available
- The part does not contain the required attributes

- Missing write permissions in the target directory (CSV File already open)

Attributes are not found
Possible causes:

- Attributes have different names
- Attributes are not stored as user attributes
- Alias and title differ

8 Toolbox in NX

The connection of the CAD system Siemens NX to the AAS enables a continuous, bidirectional flow of information. This transforms the static 3D model into an integral component of the Digital Twin. Using the NX Open API, a wide variety of functions can be implemented, expanding the functional scope for AAS use cases almost limitlessly.

The following core functions and workflows are made possible by this integration:

8.1 Automated import of CAD data from the AAS

- **AASX extraction:** CAD models (such as STEP files) referenced within an AASX container can be specifically searched for, extracted, and saved in a temporary directory using external Python scripts.
- **Automatic loading in NX:** A launcher script executed in NX picks up this data and automatically imports it into the work environment via the NX Step Importer.
- **Assemblies and single parts:** Complex assembly structures as well as individual parts can be loaded. *Note on positioning: Via the NX Open API, exact transformation matrices (coordinates) can be passed to the imported bodies during loading or within the assembly.*

8.2 Assignment of a unique Asset ID (Digital Thread)

- **ID linking:** To uniquely assign a CAD part to a specific AAS instance, a unique ID can be assigned to the model directly in NX.
- **Attribution:** This assignment is done by writing metadata into the part attributes (e.g., creating the attribute `PART_ID`). If the part is exported later, this ID is retained as meta-information within the model and can be read by Industry 4.0 systems.

8.3 Data export for Bills of Materials (BOM) and AAS synchronization

- **Reading properties:** Specific physical mass properties of an NX part (such as volume, mass, and surface area) but also user-defined attributes can be read out automatically using a Python script.
- **BOM creation:** These extracted CAD attributes can be exported (e.g., as a CSV file). This data forms the basis for detailed bills of materials (such as an R-BOM for recycling scenarios), which can then be linked directly into the structure of the AASX file.

8.4 UI customization and user interaction

- **NX Block UI Styler:** Native graphical user interfaces (dialog boxes) can be created directly in NX. For example, the user can enter IDs via a convenient text field while validation checks run in the background (e.g., whether an active part is open).
- **Seamless integration:** The NX user interface can be customized by adding scripts as interactive buttons directly to the NX menu bar (ribbon). This ensures a high level of user-friendliness.
- **Limitless extensibility:** Since NX can act both as a launcher for external Python scripts and execute internal C#/Python logic, there are virtually no technical limitations for automation and data linking with the Digital Twin.

9 Comparison of CAD Systems in the Context of the AAS

If the question arises during development as to which CAD system should be used, practical experience from various projects can be drawn upon. As shown in the following table, the three established CAD systems Siemens NX, PTC Creo, and Autodesk Fusion 360 differ from each other, particularly in terms of licenses, functional scope, and user-friendliness.

Criterion	Siemens NX	PTC Creo	Autodesk Fusion 360
Licenses	0 Siemens NX license that supports NX Open and journals	+/-/0	🚧 Work in Progress

Criterion	Siemens NX	PTC Creo	Autodesk Fusion 360
Functional Scope	+ Extensive, highly flexible via Python, but Siemens NX specific limits may apply.	+/-/0	🚧 Work in Progress
User-Friendliness	+ Through integration into the NX interface	+/-/0	🚧 Work in Progress

Table: Comparison of selected CAD systems for AAS integration

10 Execution and Integration in Siemens NX

This section describes how the developed scripts are integrated and executed within the Siemens NX environment.

The mechanisms described here apply globally to the entire project and are not repeated in individual function descriptions.

10.1 Execution Methods in Siemens NX

There are multiple ways to execute the NX launcher script inside Siemens NX.

Option 1: Journal Playback

The script can be executed using the **Journal Playback** functionality.

Steps:

- Open *Journal Playback* in NX
- Select the NX launcher script
- Execute the script

This is the simplest method and is recommended for development and testing.

Option 2: Custom User Command

The script can also be integrated as a **custom NX command**.

- A user-defined command is created in NX
- The NX launcher script is assigned as the execution action
- The command can be added to toolbars or menus

This approach is recommended for productive workflows.

10.2 File Path Dependencies

Scripts in Siemens NX are referenced via **absolute or relative file paths**.

Important considerations:

- If scripts are moved or renamed, paths must be updated
- The NX launcher script must be able to locate:
  - the external script
  - the temporary STEP file

Incorrect paths will prevent execution.

10.3 External Script Execution

The NX launcher automatically executes the external script:

- Don't change path of the downloaded scripts
- No manual interaction is required
- Communication is handled via:
  - file system (STEP file)
  - return codes
  - stdout / stderr logging

10.4 UI Styler Integration (Optional)

If a graphical interface is created using **NX UI Styler**, additional integration is possible.

Requirements

- `.cs` file (UI logic)
- `.dllx` file (compiled UI definition)

Important Notes

- The `.cs` file contains a reference to the `.dllx` file
- This path must be updated before first execution
- Both files must remain linked

Execution Options for UI Styler

UI Styler scripts can be executed via:

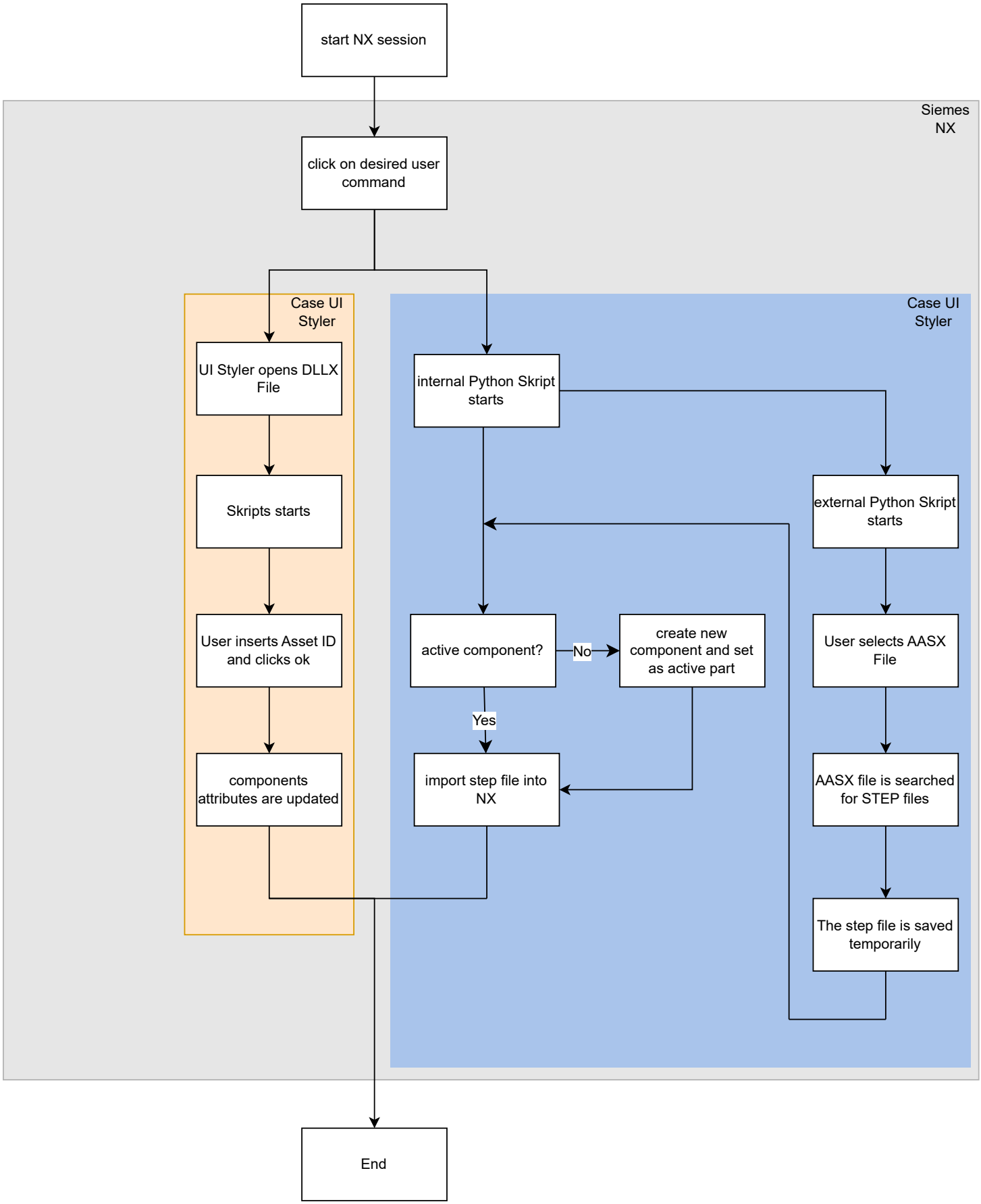
- **Journal Replay**
- **Custom NX Command**

This allows seamless integration into NX workflows.

10.5 Workflow Overview

The following diagram illustrates the interaction between:

- NX launcher script
- External AAS processing script
- Optional UI layer


start NX session
Siemes NX
click on desired user command
Case UI Styler
UI Styler opens DLLX File
Skripts starts
User inserts Asset ID and clicks ok
components attributes are updated
Case UI Styler
internal Python Skript starts
external Python Skript starts
active component?
No
create new component and set as active part
Yes
import step file into NX
User selects AASX File
AASX file is searched for STEP files
The step file is saved temporarily
End

11 Conclusion and Outlook

11.1 Conclusion

This project has successfully built a functioning bridge between the standardized AAS and the CAD system Siemens NX. In doing so, it has shown that a direct, bidirectional connection between both worlds is essential. Data must flow seamlessly throughout the entire product lifecycle – whether it is generated in CAD and transferred to the AAS, or vice versa. A clear practical example of this is the automated calculation of the material weights of an assembly in CAD, which are then made directly available for processes in the AAS.

However, there were some hurdles to overcome at the beginning of the project. One challenge was the licensing model of Siemens NX: the "NX Open" functions, which are absolutely necessary for interface programming, are missing in the free student version.

Nevertheless, we were able to successfully implement the essential tools for automated data exchange within the scope of the project:

- **Import of 3D data from the AAS:** A tool specifically extracts STEP models from AASX containers, converts them into native NX PRT files, and automatically inserts them into NX, including the option for positioning.
- **Assignment of an Asset ID to a part ("addPartID"):** Using an integrated user interface, designers can assign a unique Asset ID (`PART_ID`) to their models directly in NX. This linking of the 3D model with its AAS instance is a fundamental prerequisite for Industry 4.0.
- **Synchronize properties & Data export from CAD:** A Python script automatically reads physical properties (volume, mass, material) from NX and exports them as a CSV file – the ideal basis for recycling bills of materials (R-BOMs).
- **Seamless UI integration:** All developed functions were integrated as buttons directly into the NX menu bar (ribbon). They do not disrupt the normal design workflow and are highly intuitive to use.

11.2 Outlook

The created interfaces form a solid foundation. To completely close the data loop, the following further developments are recommended for future projects:

- **Web-based search function:** The goal is to use an Asset ID in the browser to directly download the appropriate AASX file, automatically find the contained CAD model, and seamlessly open it in NX.

- **Direct AASX export:** The current workaround using CSV files should be eliminated. In the future, material and geometry data from the CAD system should be written directly into the standardized AASX format so that it can be used in the AAS without intermediate steps.

Once this bidirectional workflow is fully established in Siemens NX, our methodology can serve as a blueprint. The concepts developed here can then be easily transferred to other CAD systems to achieve true, cross-system interoperability.

12 Tools and Sources

12.1 Tools Used

The following tools supported the implementation and documentation of the project:

- **DeepLcom:** For translations from German into English.
- **Gemini:** As an AI assistant for Markdown formatting and code snippets.
- **Drawio:** For creating the diagrams.
- **Visual Studio:** As the development environment for programming.
- **GitHub:** For version control and publishing the code.
- **Siemens NX:** For the CAD work.

12.2 Sources

The theoretical and technical basis of this project relies on the following documents:

[1] Industrial Digital Twin Association (IDTA), *Specification of the Asset Administration Shell - Part 1: Metamodel*, Version 3.0. Frankfurt am Main, Germany: IDTA e.V., 2023. [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2023/06/IDTA-01001-3-0_SpecificationAssetAdministrationShell_Part1_Metamodel.pdf

[2] European Commission, "Commission Regulation (EU) 2019/2022 of 1 October 2019 laying down ecodesign requirements for household dishwashers," *Official Journal of the European Union*, L 315, pp. 267-284, Dec. 2019.

[3] European Parliament and Council of the European Union, "Regulation (EU) 2024/1781 of 13 June 2024 establishing a framework for the setting of ecodesign requirements for sustainable products," *Official Journal of the European Union*, L, Jun. 2024.

[4] European Parliament and Council of the European Union, "Regulation (EU) 2023/1542 of 12 July 2023 concerning batteries and waste batteries," *Official Journal of the European Union*, L 191, pp. 1-117, Jul. 2023.

[5] European Parliament and Council of the European Union, "Directive 2012/19/EU of 4 July 2012 on waste electrical and electronic equipment (WEEE)," *Official Journal of the European Union*, L 197, pp. 38-71, Jul. 2012.

[6] European Parliament and Council of the European Union, "Directive 2008/98/EC of 19 November 2008 on waste and repealing certain Directives," *Official Journal of the European Union*, L 312, pp. 3-30, Nov. 2008.

[7] European Parliament and Council of the European Union, "Regulation (EC) No 1907/2006 of 18 December 2006 concerning the Registration, Evaluation, Authorisation and Restriction of Chemicals (REACH)," *Official Journal of the European Union*, L 396, pp. 1-849, Dec. 2006.

[8] Siemens Digital Industries Software, *NX Open Programmer's Guide: Block UI Styler and Journaling*, Siemens PLM Software Inc., Plano, TX, USA. [Online]. Available at: https://support.sw.siemens.com/

Apache License

Version 2.0, January 2004 http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control"

means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work.

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner.

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License.

Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License.

Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution.

You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

- **(a)** You must give any other recipients of the Work or Derivative Works a copy of this License; and
- **(b)** You must cause any modified files to carry prominent notices stating that You changed the files; and
- **(c)** You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
- **(d)** If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions.

Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions.

6. Trademarks.

This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty.

Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an **"AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND**, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE.

8. Limitation of Liability.

In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work.

9. Accepting Warranty or Additional Liability.

While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor.

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright 2026 [Christopher Wyrobek, Daniel Mohn]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

NOTICE

Anbindung CAD an Digitale Zwillinge Copyright 2026 [Christopher Wyrobek, Daniel Mohn]

This product includes software developed at
The Apache Software Foundation (http://www.apache.org/).

Third-Party Code and Dependencies

Siemens NX 2021 UI-Styler

Portions of the user interface code in this project were generated using the Siemens NX 2021 UI-Styler and subsequently modified.
Copyright © Siemens AG. All rights reserved. The use, reproduction, and distribution of this generated code and the associated NX Open libraries are subject to the end-user license agreement (EULA) of Siemens NX.

Plattform Industrie 4.0

The implementation uses the concepts of the document "Details of the Asset Administration Shell" published on www.plattform-i40.de, which is licensed under Creative Commons CC BY-ND 3.0 DE.